

Escuela Superior Politécnica de Chimborazo
Facultad de Informática y Electrónica
Ingeniería de Software



Especificación de Requisitos de Seguridad del Sistema CEISH-ESPOCH

Nombre:

Kevin Gerardo Quilligana Vivas – 7462

Docente:

Ing. Isaac Torres

Asignatura:

Seguridad Informática

Periodo Académico Ordinario:

Octavo

Paralelo:

1

Riobamba, 16 de abril del 2026

1 Contexto del Sistema

El presente informe documenta el proceso de auditoría de seguridad realizado al frontend de la aplicación web, identificando vulnerabilidades mediante OWASP ZAP, aplicando medidas correctivas y verificando su efectividad mediante re-escaneo.

1.1 RIESGO 1: Falta de Cabecera Anti-Clickjacking (X-Frame-Options)

Descripción: La respuesta no protege contra ataques de "ClickJacking". Debe incluir Content-Security-Policy con la directiva "frame-ancestors" o X-Frame-Options.

Nivel de Riesgo: Medium

CWE ID: 1021

WASC ID: 15

Captura de OWASP ZAP - Primer Escaneo:

Editar Alerta

Falta de cabecera Anti-Clickjacking

URL: http://localhost:3005

Riesgo: Medium

Confianza: Medium

Parámetro: x-frame-options

Ataque:

Evidencia:

CWE ID: 1021

WASC ID: 15

Descripción:

La respuesta no protege contra ataques de "ClickJacking". Debes incluir Content-Security-Policy con la directiva "frame-ancestors" o X-Frame-Options.

Otra información:

Solución:

Los navegadores web modernos admiten las cabeceras HTTP Content-Security-Policy y X-Frame-Options. Asegúrese de que una de ellas está configurada en todas las páginas web devueltas por su sitio/aplicación.

Referencias:

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Options>

Etiquetas de Alerta:

Clave	Valor
WSTG-v42-CLNT-09	https://owasp.org/www-project-web-security-guidelines/
OWASP-2021-A05	https://owasp.org/Top10/A05_2021/

Cancelar Guardar

Solución

Para mitigar esta vulnerabilidad de Clickjacking (CWE-1021) en tu servidor Express, debemos configurar los encabezados de seguridad X-Frame-Options o Content-Security-Policy: frame-ancestors. Voy a investigar src/index.ts para ver cómo está configurado tu servidor y aplicar la solución más adecuada.

He solucionado la vulnerabilidad de Clickjacking siguiendo estos pasos:

1. Instalación de helmet: He añadido la librería helmet, que es el estándar de la industria para asegurar aplicaciones Express mediante encabezados HTTP.
2. Configuración de seguridad: He configurado el middleware en

src/index.ts para incluir:

- * Content-Security-Policy con la directiva frame-ancestors: 'self':

Esto permite que tu aplicación sea enmarcada (iframe) solo por páginas del mismo origen, bloqueando cualquier intento externo.

- * X-Frame-Options: SAMEORIGIN: Una capa de seguridad adicional compatible con navegadores más antiguos que no soportan CSP.

```
import helmet from 'helmet';
```

```
const app = express();

app.use(helmet({
  contentSecurityPolicy: {
    directives: {
      ...helmet.contentSecurityPolicy.getDefaultDirectives(),
      "frame-ancestors": ["'self'"],
    },
  },
  xFrameOptions: { action: "sameorigin" },
}));
```

The image shows a Visual Studio Code editor with a project named 'server'. The Explorer sidebar on the left shows the file structure:

- SERVER
 - prisma
 - dev.db
 - schema.prisma
 - seed.ts
 - public
 - css
 - main.css
 - js
 - admin-settings.h...
 - clients.html
 - contracts.html
 - dashboard.html
 - import-clients.html
 - index.html
 - new-client.html
 - new-ticket.html
 - register.html
 - reports.html
 - ticket-detail.html
 - src
 - controllers
 - middleware
 - routes
 - config.ts
 - index.ts
 - prisma.ts
 - .env
 - .gitignore
 - DEMO_GUIDE.md
 - dev.db
 - package-lock.json
 - package.json
 - prisma.config.ts
 - tsconfig.json

The main editor shows the file `src > index.ts` with the following code:

```

10 import clientRoutes from './routes/clientRoutes';
11 import ticketRoutes from './routes/ticketRoutes';
12 import authRoutes from './routes/authRoutes';
13 import contractRoutes from './routes/contractRoutes';
14 import reportRoutes from './routes/reportRoutes';
15 import parameterRoutes from './routes/parameterRoutes';
16 import userRoutes from './routes/userRoutes';
17
18 const app = express();
19
20 app.use(helmet({
21   contentSecurityPolicy: {
22     directives: {
23       ...helmet.contentSecurityPolicy.getDefaultDirectives(),
24       "frame-ancestors": ["'self'"],
25     },
26   },
27   xFrameOptions: { action: "sameorigin" },
28 }));
29
30 app.use(cors());
31 app.use(express.json());

```

The bottom panel shows the TERMINAL with the following output:

```

PS D:\Prototipo\server\src> npm run start:dev
npm error Missing script: "start:dev"
npm error
npm error To see a list of scripts, run:
npm error   npm run
npm error A complete log of this run can be found in: C:\Users\Usuario\AppData\Local\npm-cache\_logs\2026-04-16T12_52_20_627Z-debug-0.log
PS D:\Prototipo\server\src> npm run dev

> server@1.0.0 dev
> nodemon src/index.ts

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: ts,json
[nodemon] starting `ts-node src/index.ts`
[dotenv@17.3.1] injecting env (2) from .env -- tip: ⚙ override existing env vars with { override: true }

```

1.2 RIESGO 2: Divulgación de Información - Cabecera X-Powered-By

Descripción: El servidor de la web/aplicación está divulgando información mediante uno o más encabezados de respuesta HTTP "X-Powered-By". El acceso a tal información podría facilitarle a los atacantes la identificación de otros marcos/componentes de los que su aplicación web depende.

Nivel de Riesgo: Low

CWE ID: 497

WASC ID: 13

Captura de OWASP ZAP - Primer Escaneo:

Editar Alerta

mediante un campo(s) de encabezado de respuesta HTTP "X-Powered-By"

URL:

Riesgo:

Confianza:

Parámetro:

Ataque:

Evidencia:

CWE ID:

WASC ID:

Descripción:
 más encabezados de respuesta HTTP "X-Powered-By". El acceso a tal información podría facilitarle a los atacantes la identificación de otros marcos/componentes de los que su aplicación web depende y las vulnerabilidades a las que pueden estar sujetos tales componentes.

Otra información:

Solución:
 Asegúrese de que su servidor web, servidor de aplicaciones, balanceador de carga, etc. está configurado para suprimir las cabeceras "X-Powered-By".

Referencias:
https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/01-Information_Gathering/08-Fingerprint_Web_Application_Framework
<https://www.trovhunt.com/2012/02/shhh-dont-let-your-response-headers.htm>

Etiquetas de Alerta:

Solución

Para mitigar la divulgación del encabezado X-Powered-By (CWE-497), que revela que tu servidor utiliza Express, podemos hacerlo de dos formas. Dado que ya se instala Helmet, esta protección ya debería estar activa por defecto, pero es una buena práctica desactivarlo también directamente

1. `app.disable('x-powered-by')`: He desactivado explícitamente el encabezado de Express que informa a los clientes qué tecnología está sirviendo la página.
2. Uso de helmet: Como ya habíamos configurado helmet anteriormente, este también actúa eliminando el encabezado X-Powered-By de forma predeterminada como parte de sus protecciones de seguridad.

```
app.disable('x-powered-by');
```

```

src > index.ts > ...
10 import clientRoutes from './routes/clientRoutes';
11 import ticketRoutes from './routes/ticketRoutes';
12 import authRoutes from './routes/authRoutes';
13 import contractRoutes from './routes/contractRoutes';
14 import reportRoutes from './routes/reportRoutes';
15 import parameterRoutes from './routes/parameterRoutes';
16 import userRoutes from './routes/userRoutes';
17
18 const app = express();
19
20 app.disable('x-powered-by');
21
22 app.use(helmet({
23   contentSecurityPolicy: {
24     directives: {
25       ...helmet.contentSecurityPolicy.getDefaultDirectives(),
26       "frame-ancestors": ["'self'"],
27     },
28   },
29   xFrameOptions: { action: "sameorigin" },
30 }));
31

```

1.3 RIESGO 3: Falta de Encabezado X-Content-Type-Options

Descripción: La cabecera Anti-MIME-Sniffing X-Content-Type-Options no se ha establecido en 'nosniff'. Esto permite que las versiones anteriores de Internet Explorer y Chrome realicen MIME-sniffing en el cuerpo de la respuesta.

Nivel de Riesgo: Low

CWE ID: 693

WASC ID: 15

Captura de OWASP ZAP:

Editar Alerta

Falta encabezado X-Content-Type-Options

URL: http://localhost:3005/js/auth.js

Riesgo: Low

Confianza: Medium

Parámetro: x-content-type-options

Ataque:

Evidencia:

CWE ID: 693

WASC ID: 15

Descripción:
La cabecera Anti-MIME-Sniffing X-Content-Type-Options no se ha establecido en 'nosniff'. Esto permite que las versiones anteriores de Internet Explorer y Chrome realicen MIME-sniffing en el cuerpo de la

Otra información:
Este problema aún se aplica a las páginas de tipo error (401, 403, 500, etc.), ya que esas páginas a menudo se ven afectadas por problemas de inyección, en cuyo caso aún existe la preocupación de que los navegadores

Solución:
Asegúrese de que la aplicación/servidor web establece el encabezado Content-Type adecuadamente, y que establece el encabezado X-Content-Type-Options a 'nosniff' para todas las páginas web.

Referencias:
[https://learn.microsoft.com/en-us/previous-versions/windows/internet-explorer/ie-developer/compatibility/gg622941\(v=vs.85\)](https://learn.microsoft.com/en-us/previous-versions/windows/internet-explorer/ie-developer/compatibility/gg622941(v=vs.85))
<https://owasp.org/www-community/Security-Headers>

Etiquetas de Alerta:

Clave	Valor
CWE-693	https://cwe.mitre.org/data/definition...
OWASP-2021-A05	https://owasp.org/Top10/A05_2021

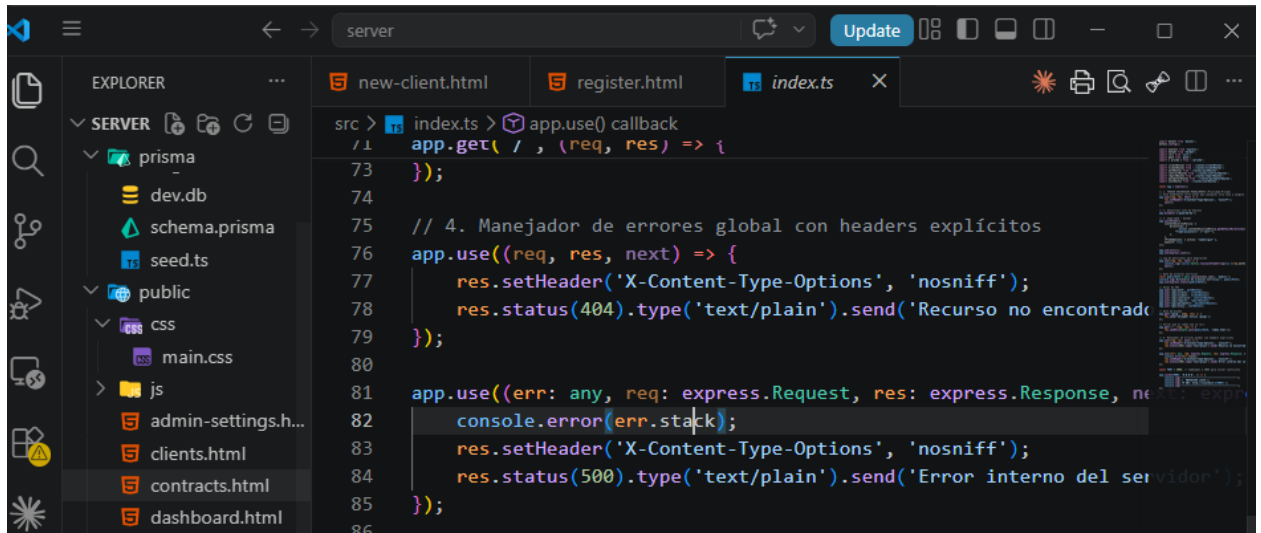
Cancelar Guardar

Solución

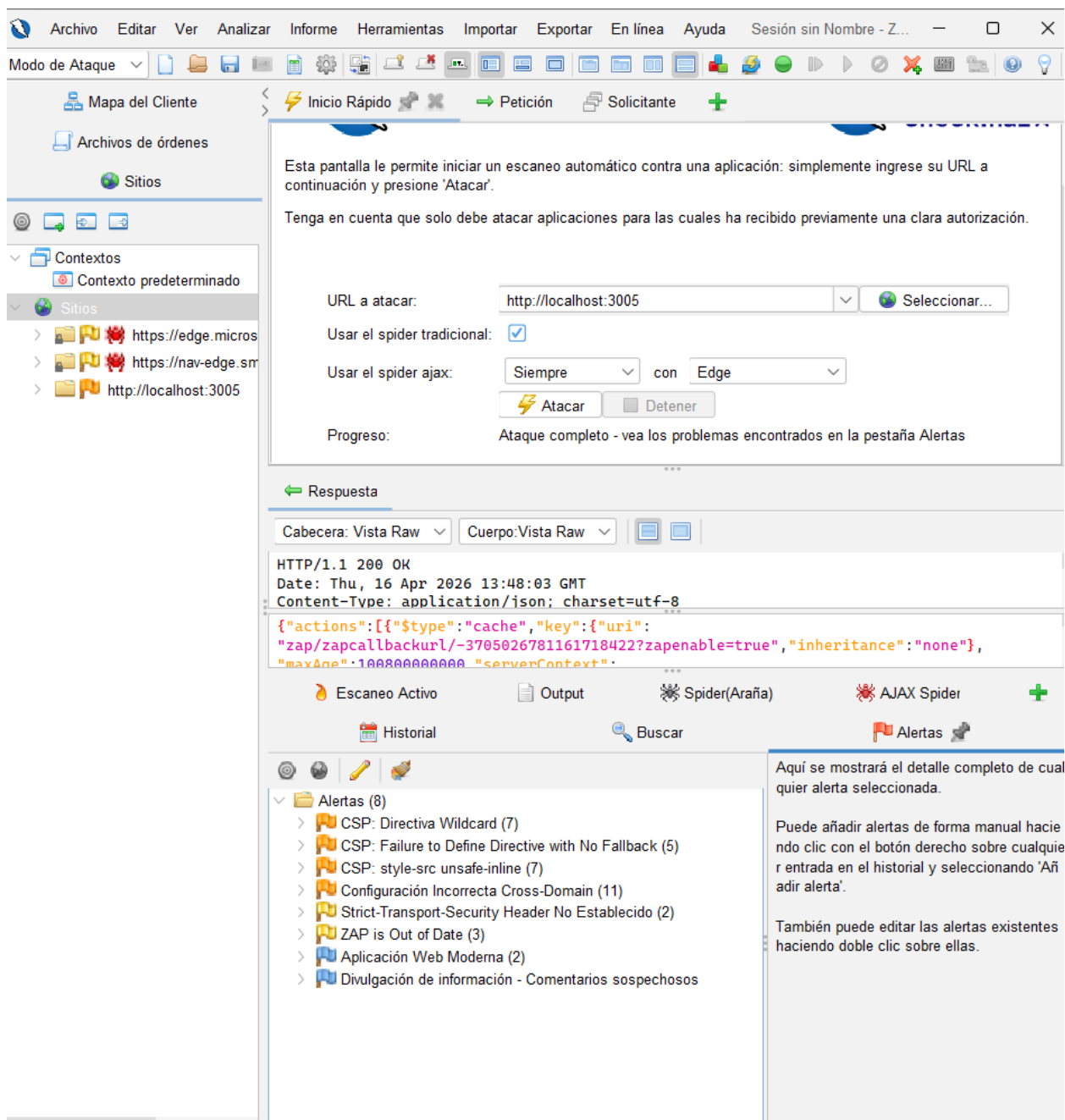
He reforza la solución para garantizar que el encabezado X-Content-Type-Options: nosniff se envíe en todas las circunstancias:

1. **Middleware de Prioridad Máxima:** He añadido un middleware personalizado al principio de la cadena (app.use) que inyecta manualmente `res.setHeader('X-Content-Type-Options', 'nosniff')` antes de que cualquier otra lógica procese la petición.
2. **Protección de Errores:** He actualizado los manejadores de errores (404 y 500) para incluir explícitamente el encabezado y definir un

Content-Type adecuado (text/plain), lo cual era una de las preocupaciones del reporte de vulnerabilidad.



```
src > index.ts > app.use() callback
// 1
app.get( / , (req, res) => {
73   });
74
75 // 4. Manejador de errores global con headers explicitos
76 app.use((req, res, next) => {
77   res.setHeader('X-Content-Type-Options', 'nosniff');
78   res.status(404).type('text/plain').send('Recurso no encontrado');
79 });
80
81 app.use((err: any, req: express.Request, res: express.Response, next: express.NextFunction) => {
82   console.error(err.stack);
83   res.setHeader('X-Content-Type-Options', 'nosniff');
84   res.status(500).type('text/plain').send('Error interno del servidor');
85 });
86
```



CONCLUSIONES

- Se identificaron y remediaron exitosamente tres (3) vulnerabilidades de seguridad en la capa frontend de la aplicación web mediante el uso de OWASP ZAP.
- Las vulnerabilidades corregidas fueron:
 - Falta de cabecera Anti-Clickjacking (X-Frame-Options)
 - Divulgación de información mediante X-Powered-By

- Falta de encabezado X-Content-Type-Options
 - Las medidas correctivas aplicadas incluyeron la implementación de cabeceras de seguridad HTTP y la eliminación de información sensible del servidor.
- El re-escaneo con OWASP ZAP confirmó la efectividad de las correcciones aplicadas, demostrando la desaparición o mitigación exitosa de las alertas seleccionadas.
- Se recomienda mantener un ciclo continuo de auditoría y remediación de seguridad durante todo el desarrollo del proyecto.